Blockchain y Criptomonedas II

UCEMA - QUANt

Clase III: 04/11/26

Matias Selser

Agenda

- Módulo X: Discusión de la tarea
- Módulo XI: *Flash Loans* y margin trading
- Módulo XII: Modelos de arbitraje para AMMs de UNIV2
- Break
- Módulo XIII: Riesgos de ejecución
- Módulo XIV: Otros productos financieros
- Anexo: monitor de arbitraje (docker compose)

Módulo X: Discusión de la tarea

10.1 Resolución de la tarea

Tarea: ante cada swap de un par (WMATIC/USDC), mostrar el precio en SushiSwap, QuickSwap, Binance y OKX.

Dos modos de suscripción (eth_subscribe):

- newHeads: un evento por bloque (~2 s en Polygon), haya o no actividad. Sirve para re-preciar todo (clase_3.py).
- logs + address + topics: solo cuando uno de nuestros pools emite Swap. Sirve para reaccionar a trades (clase_3_3.py).

La solución, paso a paso:

1. getPair en las factories de SushiSwap y QuickSwap: las direcciones de los dos pools WPOL/USDC.e.
2. Un handler: recibe el log del Swap, lo decodifica (amount0In/1In/0Out/1Out) y lee los 4 precios (reservas de los pools, y POL/USDT por CCXT en Binance y OKX).
3. Alimentarlo: eth_getLogs de los swaps recientes, y una suscripción por websocket a los logs de los dos pools.

En vivo: notebook clase_3.ipynb, sección 2 (WPOL/USDC.e: ~100 swaps por hora entre los dos DEX).

Módulo XI: *Flash loans* y *Margin trading*

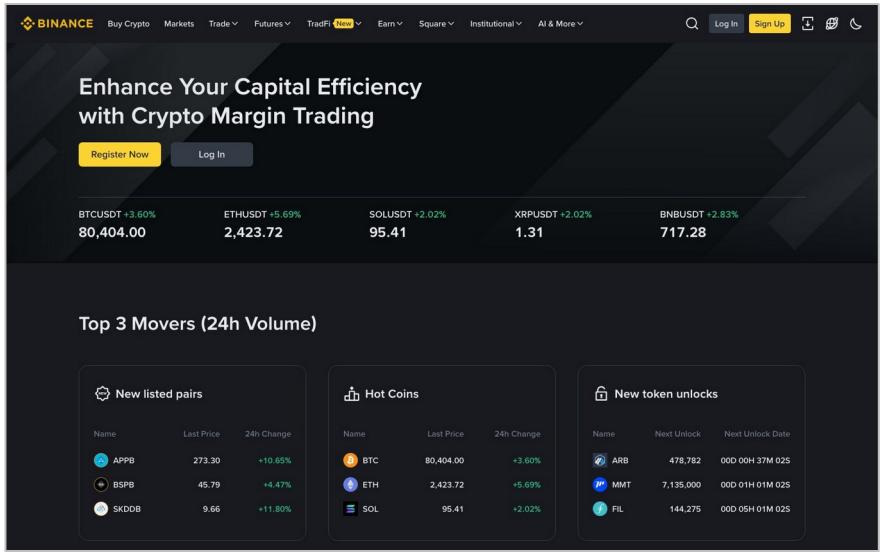
11.1 ¿Cómo podemos hacer funcionar estas ideas?

- Restricción: custodia de tokens
  - Cexes en general son *custodials* de los tokens, mientras que los dexes no lo son
- Principal restricción: manejo de capital.
- Para hacer un arbitraje cex-dex, habría que disponer de tokens en un mercado y usar esos tokens en el otro, todo en el mismo espacio temporal.
- Complejo operativamente

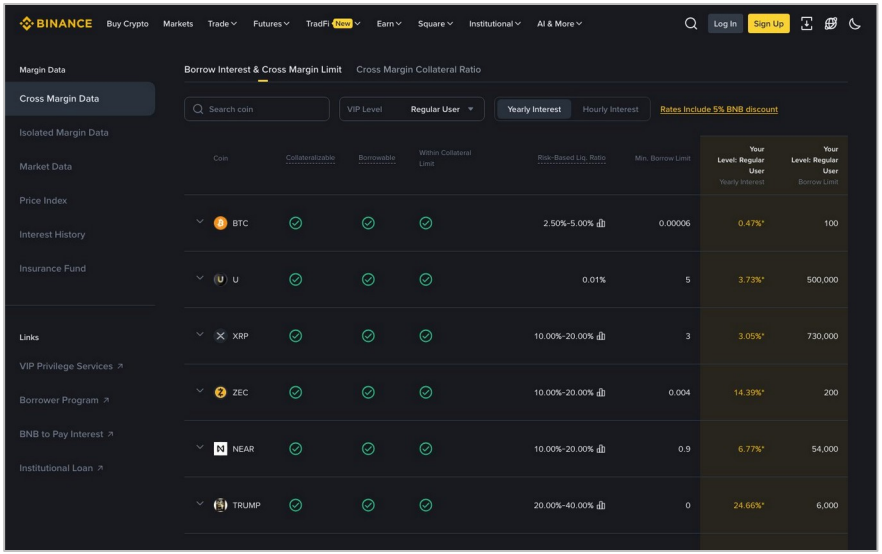
11.2 Cexes: *Margin Trading*

- Cada cex tiene una forma distinta de implementarlo, pero la idea es la siguiente
- Se collateraliza una posición con algún token (puede ser una stable coin, por ejemplo) y se opera el token deseado
- Referencias: binance

En la práctica: *Binance*



Margin trading en Binance
https://www.binance.com/en/margin-trading

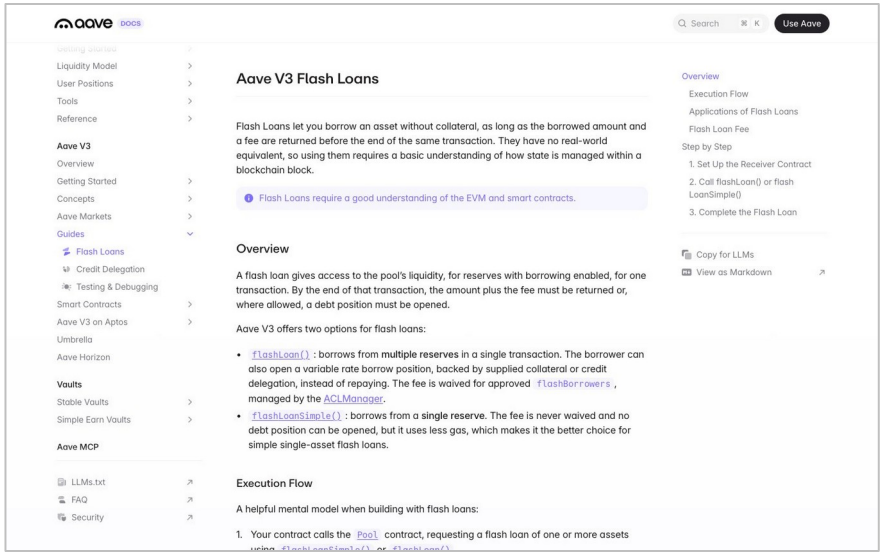


Tasas y límites de margin
https://www.binance.com/en/margin-fee

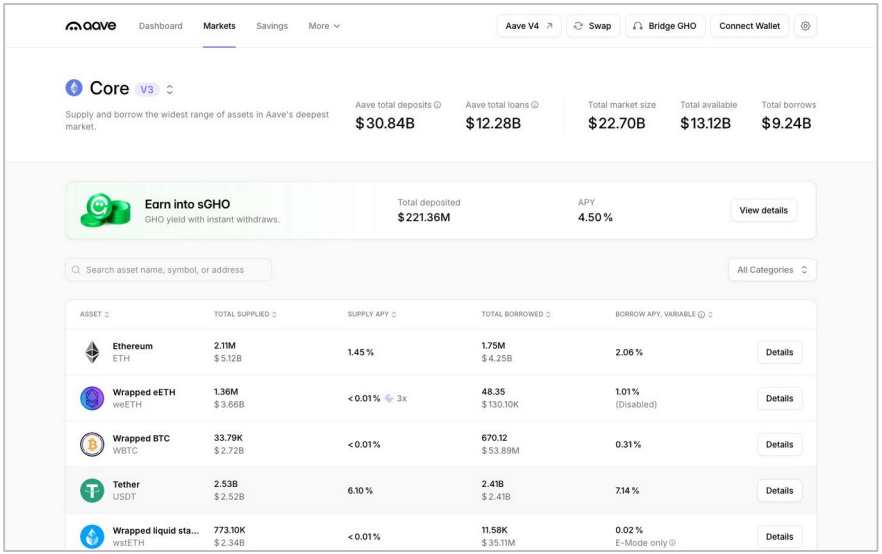
11.3 *Flash Loans* o *Borrow*

- En defi, las opciones son un poco más amplias.
- *Flash loans*:
  - Préstamos intra transacción. Permiten pedir prestado activos sin collateralizar nada, con la condición de que se devuelvan en la misma transacción
  - Solo implementables con smart contracts
- Borrowing:
  - Existen protocolos como AAVE o Compound que permiten pedir préstamos sobrecollateralizados de tokens.
  - Más transacciones, más complejidad para cerrar el círculo

En la práctica: *Aave*



Docs: flash loans en Aave V3
https://aave.com/docs/aave-v3/guides/flash-loans

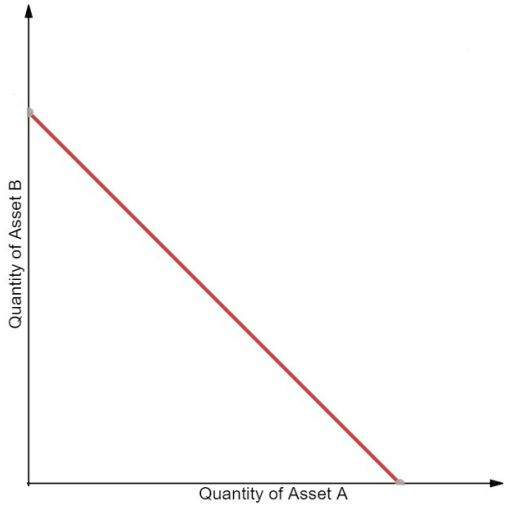


Mercados de Aave: supply y borrow
https://app.aave.com/markets/

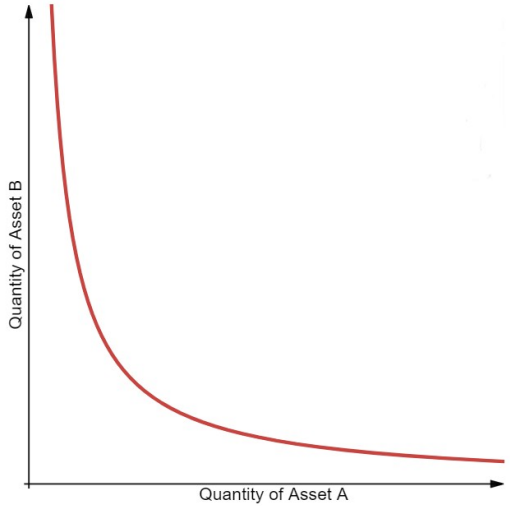
Módulo XII: Modelos de arbitraje para AMMs de UNIV2

12.1 *CPMM* y *CSMM*

- *CSMM*: *constant sum market maker*
  - Consiste en un modelo en el cual la suma de los tokens es constante: $x + y = k$

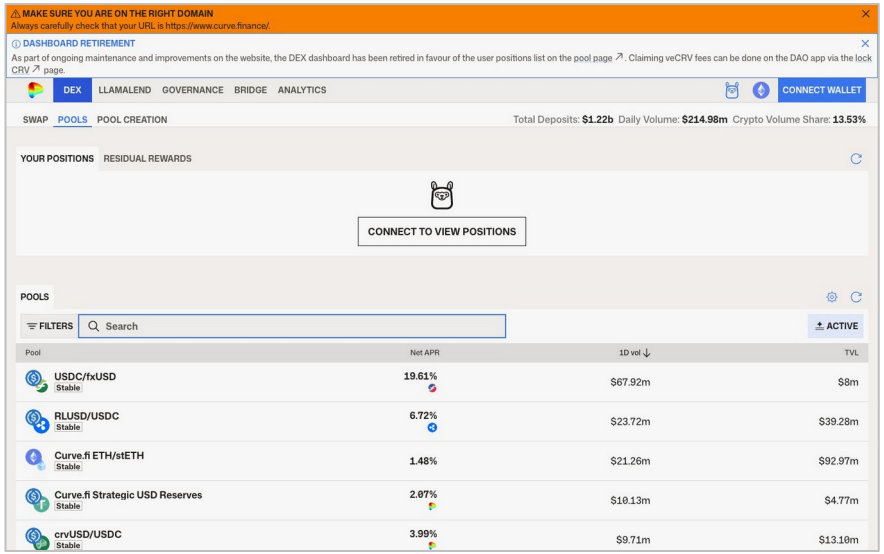


- *CPMM*: *constant product market maker*
  - Consiste en un modelo en el cual el producto de los tokens es constante: $xy = k$

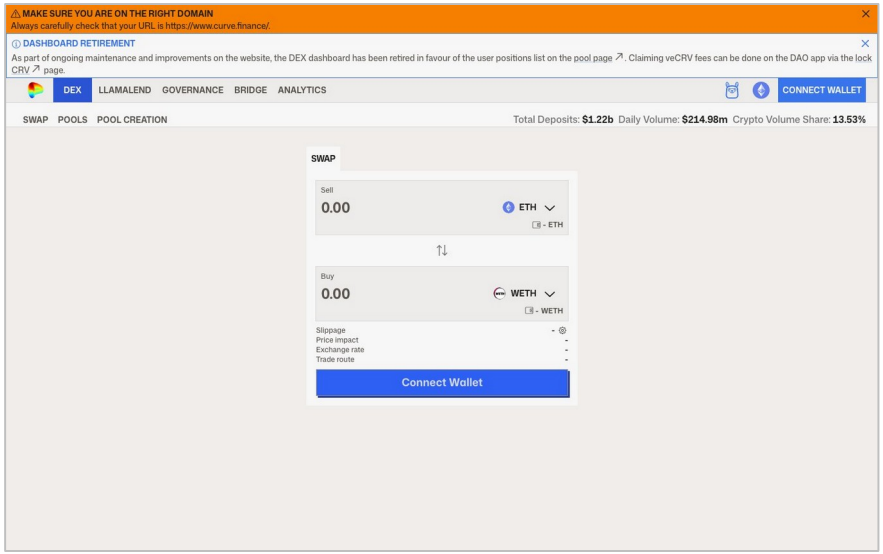


- *CPMM vs CSMM:*
  - *CPMM* tiene liquidez infinita
  - CSMM permite drenar las reservas, mal modelo
    - usado en stable coins en algunos casos
  - Problemas de esta generación:
    - *Impermanent loss*: recordemos, diferencia entre quedarse el token y e invertirlo en un AMM. Producido por la divergencia hacia algún extremo del pool
    - Baja eficiencia de capital:
      - Se necesitan muchos tokens para generar un *price impact* similar al observado en los *orderbooks* tradicionales.
      - Provee liquidez en todo el rango de precios, y no en el rango de precios trascendente (revisar UNI V3)

En la práctica: *Curve (stableswap)*



Pools de Curve
https://www.curve.finance/dex/ethereum/pools



Swap en Curve
https://www.curve.finance/dex/ethereum/swap

12.2 Maximización

- Volviendo al caso implementado en múltiples exchanges, se pueden observar divergencias en los precios absolutos de los activos
- ¿Son aprovechables? Veamos
- Recordemos:
  - Pools uni v2, CPMM
  - Para un par determinado, en un exchange determinado: $x_1 y_1 = k_1$
  - Para el mismo par, en otro exchange: $x_2 y_2 = k_2$
- Para todos los pares, podemos obtener (y ya lo hicimos!)
  - La dirección del pool asociado
  - Las reservas de ambos tokens, entonces podemos obtener k

- ¿Qué pasa si intentamos agregar una cantidad dx al pool?

$$(x + dx)(y - dy) = k$$

- Para que el producto se mantenga constante, al agregar una cantidad dx vamos a sacar una cantidad dy. ¿Y cómo calculamos dy?

$$(y - dy) = \frac{k}{x + dx}$$

$$y - \frac{k}{x + dx} = dy$$

- ¡Determinístico! Excelente. Qué pasa si, entonces, intentamos encadenar dos pools entre sí. Es decir, colocar dx1 en el primer pool, obtener dy1, y poner dy1 en el segundo pool para obtener dx2.

$$y_1 - \frac{k_1}{x_1 + dx_1} = dy_1$$

$$(x_2 - dx_2)(y_2 + dy_1) = k_2$$

$$(x_2 - dx_2)\left(\frac{(y_1 + y_2)(x_1 + dx_1) - k_1}{x_1 + dx_1}\right) = k_2$$

- Entonces, dx2 es igual a

$$(x_2 - dx_2) = \frac{k_2\,(x_1 + dx_1)}{(y_1 + y_2)(x_1 + dx_1) - k_1}$$

$$x_2 - \frac{k_2\,(x_1 + dx_1)}{(y_1 + y_2)(x_1 + dx_1) - k_1} = dx_2$$

Usando $k_1 = x_1 \cdot y_1$ y $k_2 = x_2 \cdot y_2$, se simplifica a:

$$dx_2 = \frac{x_2\,y_1\,dx_1}{x_1 y_2 + (y_1 + y_2)\,dx_1}$$

- Por lo tanto, ¿cuál sería el problema que estamos buscando optimizar?
  - ¡Obtener la mayor cantidad de tokens a la salida!

$$argmax_{dx_1} \; F(x_1, y_1, x_2, y_2, dx_1)$$

$$F(x_1, y_1, x_2, y_2, dx_1) = dx_2 - dx_1$$

- ¿Cómo resolver?
  - ¡Podemos derivar con respecto a dx1 y hallar ese valor!
  - Resolución: en la slide siguiente (y en la notebook, con fees)

Resolución: el óptimo sin fees

Con $k_1 = x_1 \cdot y_1$ y $k_2 = x_2 \cdot y_2$, lo que ganamos es $F(dx_1) = dx_2 - dx_1$:

$$F(dx_1) = \frac{x_2\, y_1\, dx_1}{x_1 y_2 + (y_1 + y_2)\, dx_1} - dx_1$$

Derivando con respecto a dx_1 e igualando a cero:

$$F'(dx_1) = \frac{x_1\, y_1\, x_2\, y_2}{\left(x_1 y_2 + (y_1 + y_2)\, dx_1\right)^2} - 1 = 0$$

Solución:

$$dx_1^* = \frac{\sqrt{x_1\, y_1\, x_2\, y_2} - x_1\, y_2}{y_1 + y_2}$$

Hay oportunidad solo si $dx_1^* > 0$, es decir $x_2 \cdot y_1 > x_1 \cdot y_2$. Con fees ($\gamma = 0{,}997$) la estructura es la misma: notebook, sección 5.

Generalización: ciclos de n pools

Un ciclo A → B → … → A de n pools equivale a un solo pool virtual. Con (E_in, E_out) = (r_1_in, r_1_out), para cada pool i = 2 … n:

$$E_{in} \leftarrow \frac{E_{in}\, r_i^{in}}{r_i^{in} + \gamma\, E_{out}}, \qquad E_{out} \leftarrow \frac{\gamma\, E_{out}\, r_i^{out}}{r_i^{in} + \gamma\, E_{out}}$$

El ciclo entero es un solo swap, y el óptimo tiene la misma forma cerrada:

$$\mathrm{out}(dx) = \frac{\gamma\, E_{out}\, dx}{E_{in} + \gamma\, dx} \qquad dx^* = \frac{\sqrt{\gamma\, E_{in}\, E_{out}} - E_{in}}{\gamma}$$

Hay oportunidad si y solo si:

$$\gamma^n \prod_{i=1}^{n} \frac{r_i^{out}}{r_i^{in}} > 1$$

Con n = 2 y γ = 1 es la fórmula anterior. Ejemplo real triangular (USDC.e → WETH → WPOL → USDC.e en SushiSwap y QuickSwap): notebook, sección 5d.

Uniswap V3: liquidez concentrada

Cada LP elige un rango de precios [Pa, Pb] y el pool guarda √P y la liquidez activa L. Dentro del rango actual, un pool V3 es un pool V2 con reservas virtuales:

$$x_v = \frac{L}{\sqrt{P}}, \qquad y_v = L\sqrt{P}, \qquad x_v\, y_v = L^2$$

La fórmula de swap de V2 vale igual (γ = 1 − fee: 0,05%, 0,3% o 1%):

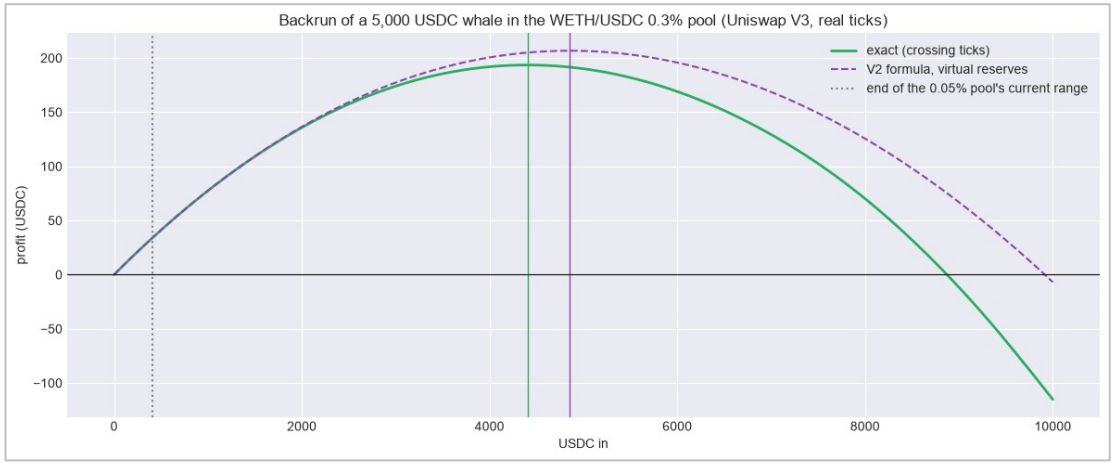
$$dy = \frac{\gamma\, dx\, y_v}{x_v + \gamma\, dx}$$

… pero solo hasta el próximo tick inicializado. Vendiendo token0, lo máximo que entra en el rango es:

$$\Delta x_{max} = \frac{L}{\gamma}\left(\frac{1}{\sqrt{P_b}} - \frac{1}{\sqrt{P}}\right)$$

Polygon hoy (WETH/USDC): el pool 0,05% tiene reservas virtuales de 2,27 M USDC, pero en su rango actual solo entran ~400 USDC; el 0,3%, ~116 USDC. Pasado el tick, L cambia y el trade sigue en el rango siguiente.
Simulador tramo por tramo validado contra el QuoterV2 de Uniswap en el mismo bloque (error ~1e-8): notebook, sección 5e.

V3: arbitraje cruzando ticks (backrun)



Una ballena compra WETH en el pool 0,3% (queda caro); el arbitrajista compra en el 0,05% y vende en el 0,3%.

dx* fórmula (virtual) vs exacto (USDC), hoy:
ballena 2.000: 1.707 = 1.707
ballena 5.000: 4.854 vs 4.405
ballena 20.000: 12.209 vs 15.803

Dentro del rango la fórmula es exacta; si el trade cruza ticks es solo una primera aproximación.

Receta para cualquier ciclo que mezcle V2 y V3: reservas virtuales en los tramos V3, un γ_i por tramo, y plegar:

$$E_{in} \leftarrow \frac{E_{in}\, r_i^{in}}{r_i^{in} + \gamma_i\, E_{out}}, \qquad E_{out} \leftarrow \frac{\gamma_i\, E_{out}\, r_i^{out}}{r_i^{in} + \gamma_i\, E_{out}}$$

La condición de oportunidad $\gamma_1 \ldots \gamma_n \cdot \prod$ r_out/r_in > 1 es exacta también en V3 (depende solo de precios marginales). Si dx* cruza ticks: maximizar la ganancia exacta, que sigue siendo cóncava, con una búsqueda 1-D.

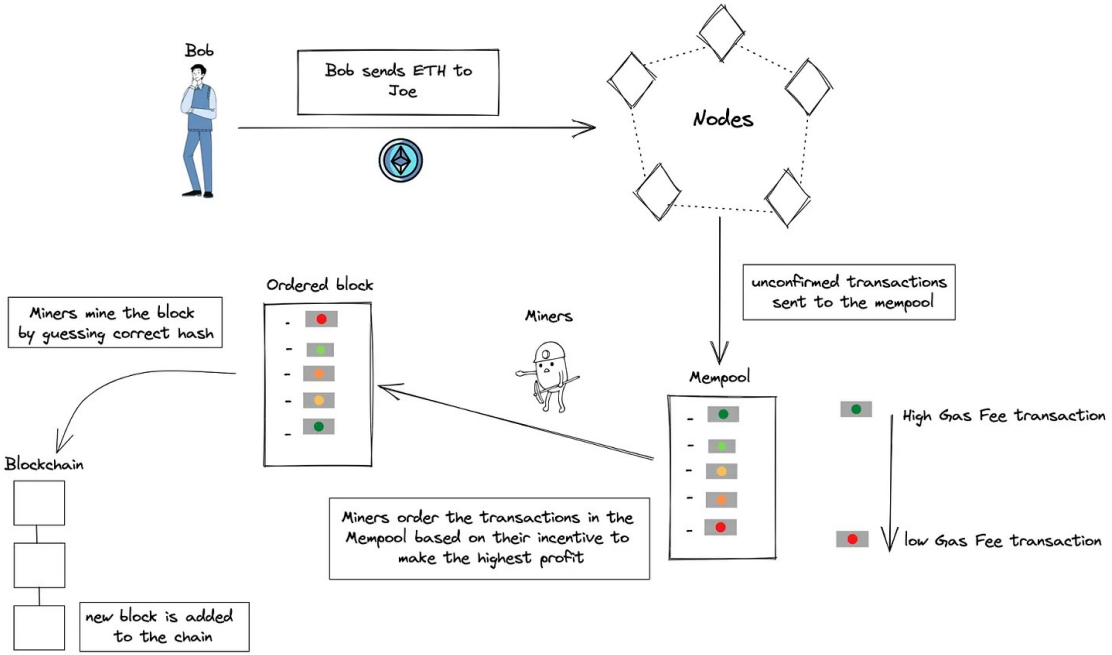
12.3 ¿Qué pasaría si…?

- Detalles a tener en cuenta respecto al setup:
  - *Fees*:
    - ¿Se están considerando los fees de *trading* de los pooles en este set up?
    - ¿Cómo cambian las ecuaciones si los fuéramos a considerar?
  - Restricciones:
    - En el problema original, no está acotado el dominio de las variables asociadas
    - ¿Qué pasa si tenemos en cuenta que, por ejemplo, tenemos un máximo número de tokens para operar? Es decir, nuestra *wallet* tiene 1 eth y no más para poner en el pool. Restricciones de capital
  - Ecuaciones:
    - En este caso, este problema se puede plantear porque conocemos el comportamiento de las curvas. ¿Qué se puede hacer si las ecuaciones no son derivables?
    - ¿Y si en vez de un pool tenemos un *orderbook* involucrado?

Módulo XIII: Riesgos de ejecución

13.1 *Mempool*

- ¿Cómo es el flujo de desarrollo de una transacción en Ethereum?



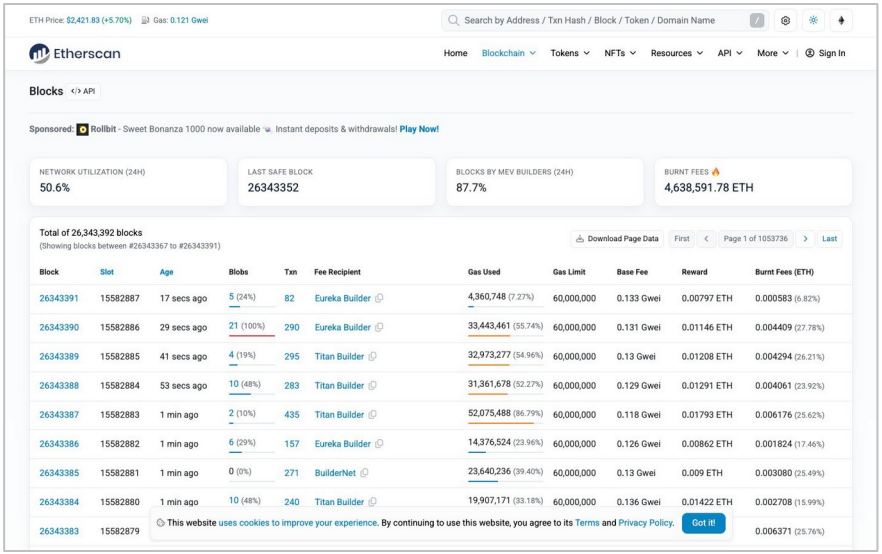
- Usuario envía la transacción a su nodo
- El nodo comparte la transacción al resto de los nodos
- Las transacciones se ordenan al armar el bloque (hoy: builders, vía MEV-Boost)
- El validador elegido propone el bloque; ~90% de las veces, el de un builder vía MEV-Boost (estudio 2024)
- El bloque se propaga a la red

- Ahora bien, ¿cómo se arma el bloque?
  - ¡Los validadores tienen un rol activo en esa decisión!
  - Si uno fuera dueño de un nodo, cuando gana el sorteo podrá elegir de qué forma se ordenan las transacciones
  - Por ejemplo:
    - Se observa que en la mempool alguien está intentando hacer un swap entre el token A y el token B
    - Cómo se conocen los parámetros del *swap*, es fácil (dado que la ecuación es determinista) conocer el resultado de las reservas después de esta transacción
    - Resolviendo entonces el problema anterior, se podría computar cual es el *input* óptimo para ese pool que se desarbitra para maximizar la ganancia! (si es que hay)
    - Se envía entonces un “doble swap” atrás del swap que desarbitra las reservas

En la práctica: *MEV-Boost y Etherscan*



MEV-Boost: bloques armados por builders
https://mevboost.pics/



Etherscan: bloques y su fee recipient
https://etherscan.io/blocks

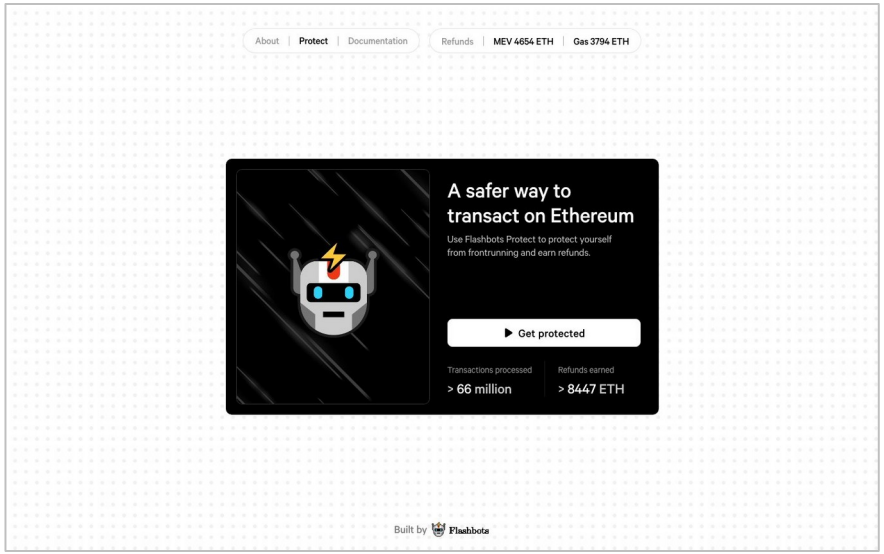
- Si fuera tan sencillo…
  - Muy bien, entendemos cómo funciona el arbitraje entre pools
  - Si es tan sencillo de implementar, ¿qué dificultades se pueden llegar a presentar para tomar esos arbitrajes? ¿Ideas?
    - ¡Sistema de lotería! Si controlamos un nodo, solo vamos a poder ordenar las transacciones con muy baja frecuencia (proporcional al *share* de tokens *stakeados* que tengamos)
    - Si usamos un *provider*, quizás tengan un mejor share. Pero competimos por el mismo arbitraje con el resto de los arbitradores. Para obtener ese arbitraje, tenemos que mejorar el gas price o prima para el validador para que nuestra transacción se ordene como queremos
    - Por otro lado, este modelo de arbitraje es intensivo en capital. Se necesita tener ambos tokens para poder hacer el *swap*. Esto suele implicar hundir capital, y tiene un costo de oportunidad asociado

- Si fuera tan sencillo… (continuación)
  - Asumamos que podemos resolver lo anterior asociado al capital, y tenemos suerte de que podemos ordenar nuestra transacción en el lugar adecuado del bloque. ¡En realidad necesitamos hacer dos transacciones! Un swap en el primer pool, y otro swap en la dirección contraria en el segundo pool.
    - Solución a esto: smart contract que en una transacción haga los dos swaps! Se puede resolver en Remix
  - Si no hay disponibilidad de capital, hay que hacer un borrow. Otra transacción.
  - Infraestructura…

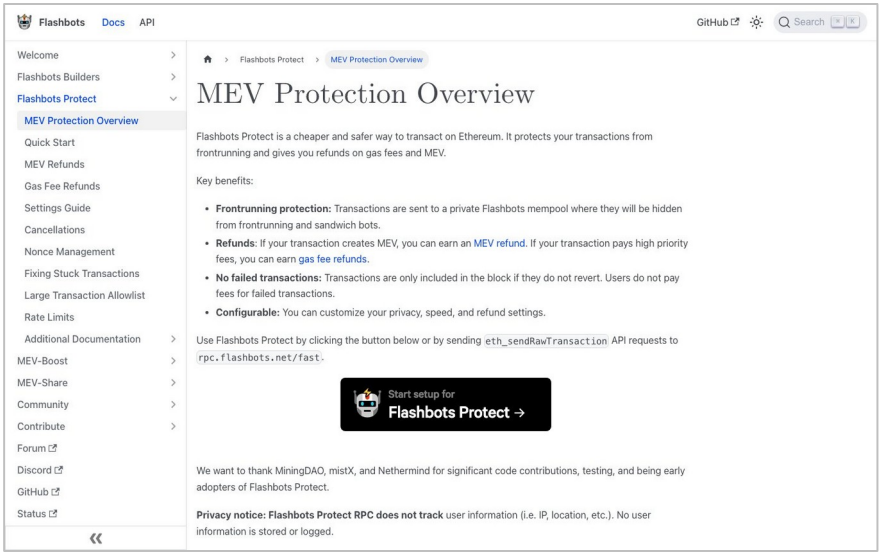
13.2 Privacidad

- Existen providers de mempool que nos permiten tener acceso a las transacciones pendientes de los distintos usuarios
- ¿Todas las transacciones son públicas?
- ¡No! Existen servicios de nodos privados, sobre los cuales no se tiene acceso a la mempool
- Flashbots Protect, MEV Blocker: RPCs privados que NO publican la transacción en la mempool. ¡Más seguro!

En la práctica: *Flashbots Protect*



Flashbots Protect: RPC privado
https://protect.flashbots.net/



Docs de Flashbots Protect
https://docs.flashbots.net/flashbots-protect/overview

13.3 Ataques

- Como vimos anteriormente, existe un parámetro en los swaps del modelo de UNIV2 que se llama *minAmountOut*. Esto responde pura y exclusivamente a la necesidad de proteger los trades de los usuarios.
- Si uno fuera a enviar un trade desprotegido (*minAmountOut* = 0), podría quedar expuesto a *frontrunning*, *backrunning*, o *sandwich trading*.

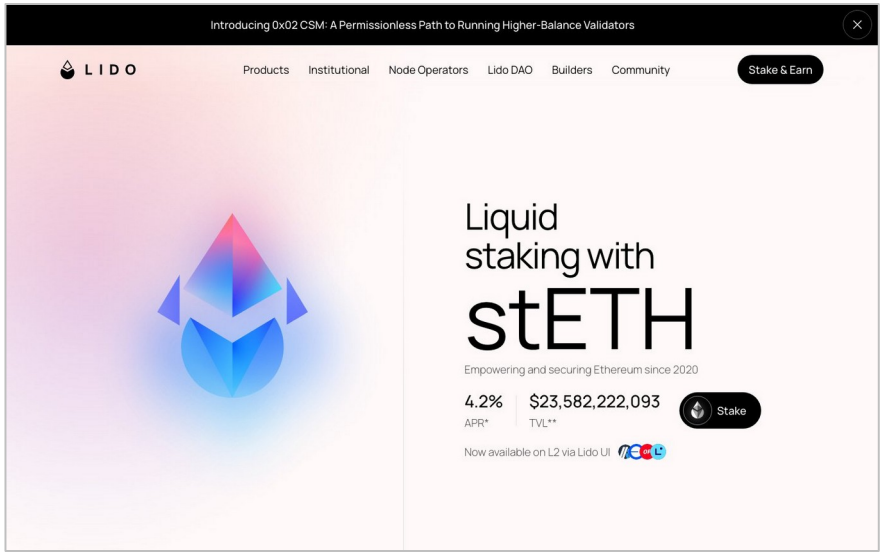
- *Frontrunning*
  - Consiste en ver una transacción pendiente, procesarla, y enviar una transacción para que se ejecute antes que esa
    - Ejemplo: liquidaciones
- *Backrunning*
  - Consiste en ver una transaccion pendiente, procesarla, y enviar una transacción para que se ejecute justo después que esa
    - Ejemplo: arbitraje
- *Sandwitch trading*
  - Consiste en ver una transacción pendiente, procesarla, y enviar una transacción para que se ejecute justo antes y otra que se ejecute justo despúes
    - Ejemplo: *riskless profit* de un swap desprotegido

Módulo XIV: Otros productos financieros

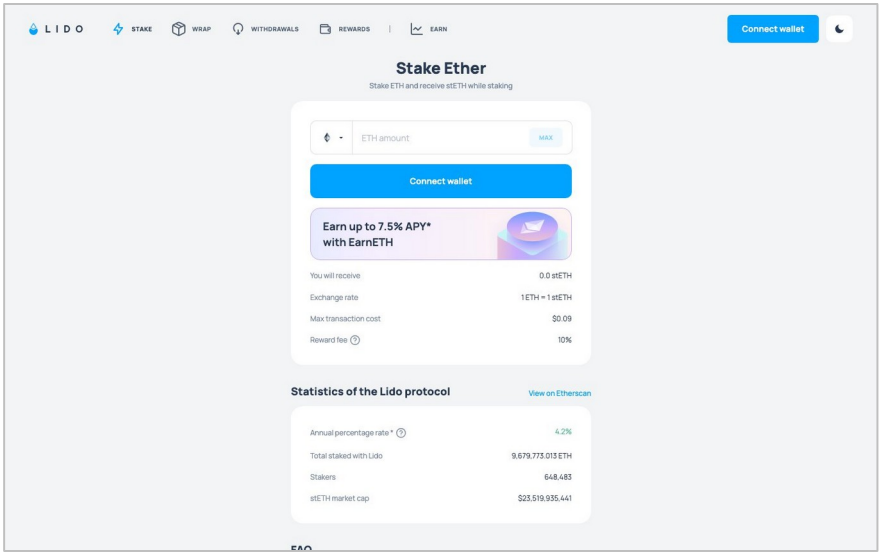
14.1 Staking

- Consiste en:
  - Tomar un token determinado
  - Colocarlo en un depósito
  - A cambio de un rendimiento, puede ser fijo o variable, en tokens
- Incentiva:
  - Al usuario, porque recibe tokens simplemente por tener tokens
  - Al protocolo, porque es una estrategia de incremento de usuarios
- Desincentiva:
  - Al trading excesivo: funciona como una manera de reducir el circulante en el ambiente
  - Al usuario, porque sus tokens están bloqueados (a veces por períodos fijos)
  - Al protocolo, porque “debe emitir” para cubrir los intereses a pagar

En la práctica: *Lido (staking)*



Lido: liquid staking (stETH)
https://lido.fi/

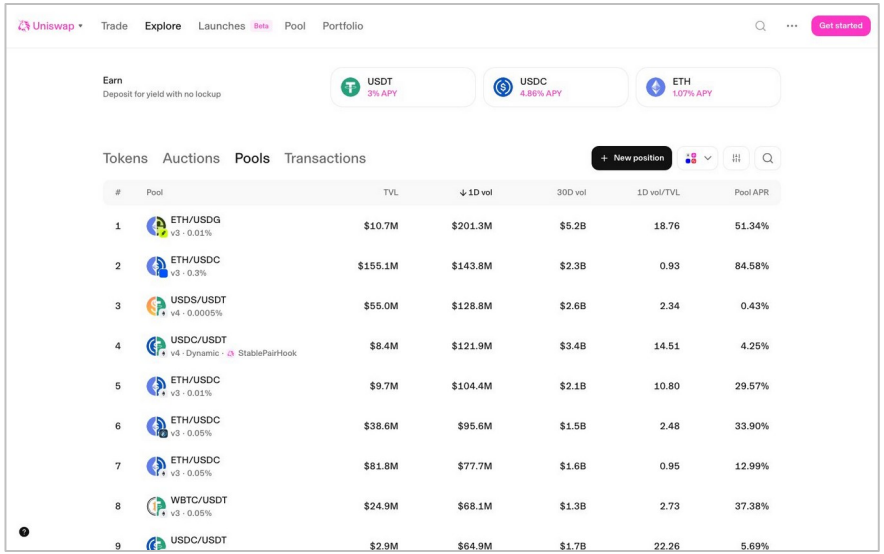


Stake de ETH en Lido
https://stake.lido.fi/

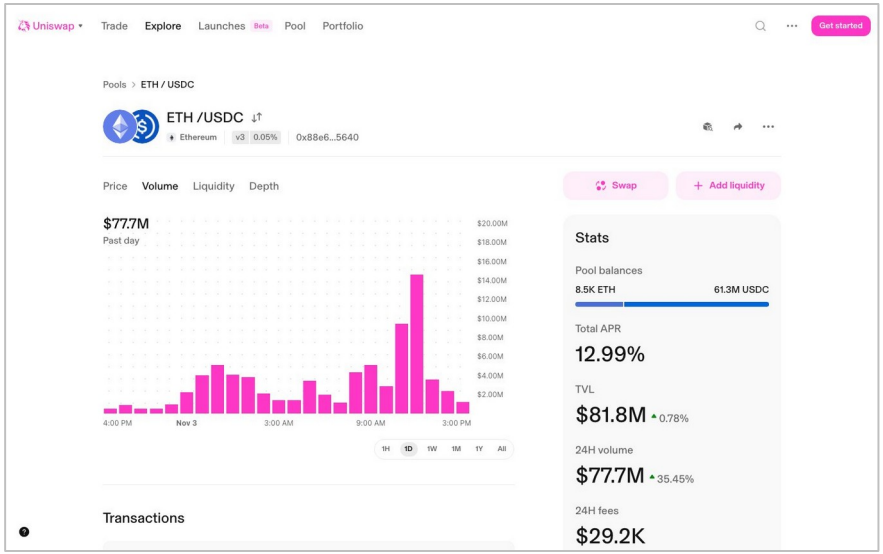
14.2 *Liquidity providing*

- Consiste en “hacer market making” en los pools
- Uno deposita tokens en una proporción determinada (depende del punto de operación del pool) y recibe los fees prorrateados por la proporción de la liquidez que representa.
- *Impermanent loss*
- Dependiendo el modelo de pool, la liquidez puede ser concentrada (Uniswap V3/V4), o distribuida (Uniswap V2) entre otros modelos

En la práctica: *Uniswap (liquidity providing)*



Pools de Uniswap
https://app.uniswap.org/explore/pools

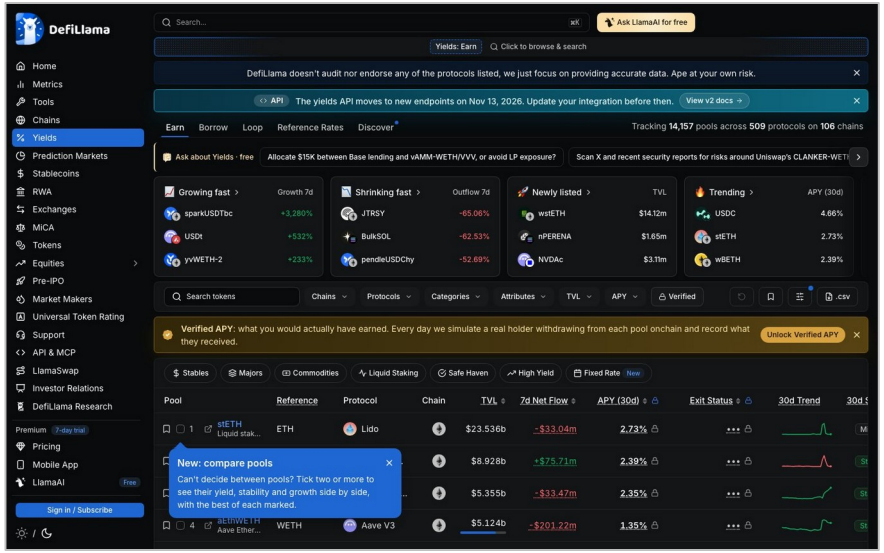


Pool V3 ETH/USDC: liquidez y volumen
https://app.uniswap.org/explore/pools/ethereum/0x88e6A0c2dDD26FEEb64F039a2c41296FcB3f5640

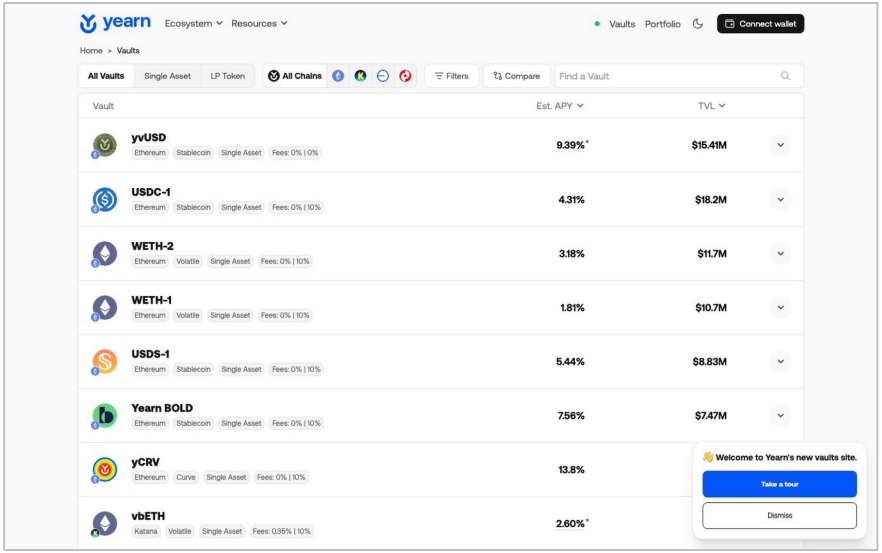
14.3 *Yield Farming*

- Consiste en:
  - Tomar un token de liquidity providing
  - Stakear ese token de liquidity providing para obtener mayor share del pool
- Sirve para:
  - Aumentar la liquidez en los pools, ya que incentiva a los usuarios a ofrecer liquidez
  - También a mantener a los usuarios ofreciendo liquidez por más tiempo
- Riesgos:
  - Nuevamente, *impermanent loss*
  - Muy competitivo. ¡Requiere mucho seguimiento!

En la práctica: *DefiLlama y Yearn (yield farming)*



DefiLlama: rankings de yields
https://defillama.com/yields

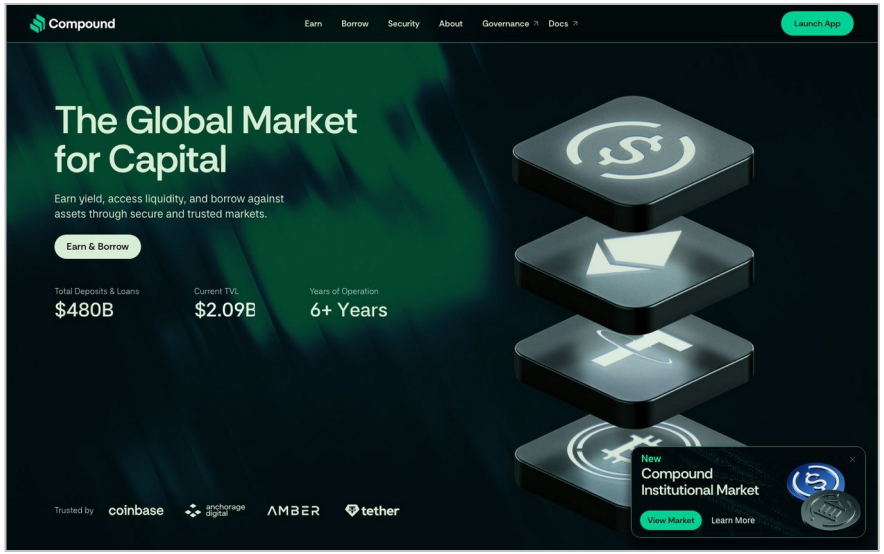


Vaults de Yearn
https://yearn.fi/vaults

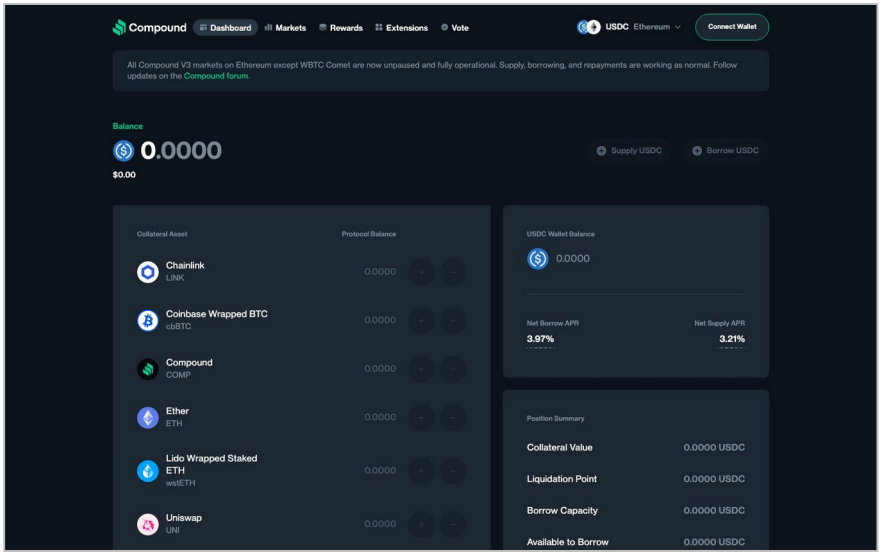
14.4 *Lending / Borrowing*

- Existen protocolos como Aave, Compound, Morpho o Spark que permiten tomar préstamos en tokens.
- Son préstamos sobrecolateralizados
- También permiten prestar los tokens para obtener un rendimiento
- Se puede integrar esta idea a smart contracts, trading, etc.

En la práctica: *Compound (lending)*



Compound: lending y borrowing
https://www.compound.xyz/



Mercados de Compound
https://app.compound.xyz/

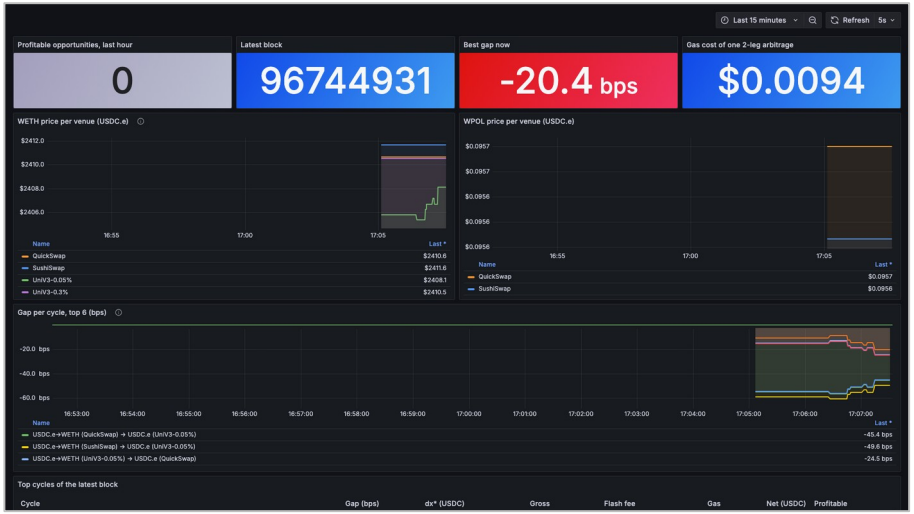
Anexo
Monitor de arbitraje con docker compose

Monitor de arbitraje: el stack

collector: en cada bloque lee 8 pools (SushiSwap y QuickSwap V2, Uniswap V3) en una sola llamada Multicall3 → tabla pool_states
detector: evalúa 30 ciclos por bloque (2 pools y triangulares) con la fórmula cerrada → tabla opportunities (gap, dx*, neto con gas y flash loan)
postgres · grafana (dashboard en vivo) · **adminer** (SQL)

cd modulo-2/clase_3/docker
docker compose up -d --build
abrir http://localhost:3030 (sin login)

Instrucciones: modulo-2/clase_3/docker/INSTRUCCIONES.md



Conclusiones y resumen

Resumen

- Módulo X: Discusión de la tarea
  - Modos de suscripción, nuevo bloque y evento
- Módulo XI: *Flash Loans* y margin trading
  - ¿Cómo resolvemos la cuestión operativa de tomar estos arbitrajes?
- Módulo XII: Modelos de arbitraje para AMMs de UNIV2
  - ¿Cómo resolvemos la cuestión matemática de tomar estos arbigrajes?
- Módulo XIII: Riesgos de ejecución
  - No es tan fácil como parece…
- Módulo XIV: Otros productos financieros
  - Distintas ideas del mundo defi

Conclusión

- Existen limitaciones para operar entre el mundo cex y el mundo dex
- Pudimos implementar casi end to end un sistema de monitoreo de arbitrajes, tanto entre dexes, entre cexes, y cex-dex
- Discutimos riesgos operativos asociados
- Revisión general de otros productos financieros, para quien tenga interés



¡Muchas gracias!